

ReconForge FAQ

Practical answers to common questions, troubleshooting steps, and operational guidance.

Installation & Setup

Q: I get “Tool not found” errors when running a module

A: ReconForge wraps external tools — they must be installed separately. When a tool is missing, you’ll see a `ToolNotFoundError` in the log and the phase that uses it will produce empty results.

Check which tool is missing from the error message, then install it using the command from `config/tools.yaml`. Common ones:

```
# Network tools
sudo apt install -y nmap enum4linux smbclient ldap-utils hydra

# AD tools
pip install enum4linux-ng impacket bloodhound
pipx install netexec

# Web tools
sudo apt install -y nikto gobuster sqlmap
gem install whatweb wpscan
pip install wafw00f
go install github.com/ffuf/ffuf@latest
go install github.com/projectdiscovery/nuclei/v2/cmd/nuclei@latest

# API tools
go install github.com/projectdiscovery/httpx/cmd/httpx@latest
pip install arjun
```

To verify a tool is available:

```
which nmap # should return a path
```

Q: Which tools are actually required vs optional?

A: Only `nmap` is marked as `required: true` in `config/tools.yaml`. All other tools are optional. If an optional tool is missing, its phases gracefully degrade — you get fewer results, not a crash.

Q: I get permission errors running nmap SYN scans

A: SYN scans (`-sS`) and UDP scans require root privileges. Either:

1. Run with sudo: `sudo python reconforge.py network --target 10.10.10.1`
2. Or the framework falls back to connect scans (`-sT`) which don’t require root but are noisier

Q: How do I install the cryptography package for encrypted loot?

A:

```
pip install cryptography
```

Without this package, using `--encrypt-loot` emits a warning and loot is saved in plaintext. The framework does not crash.

Q: Where are configuration files located?

A: Configuration files are in the `config/` directory at the project root:

- `config/tools.yaml` — Tool paths, timeouts, scan profiles, install commands
- `config/profiles.yaml` — OPSEC profiles (stealth, normal, aggressive) with timing and technique toggles

The `ConfigLoader` class (`core/config_loader.py`) reads these. You can override the config directory by passing `config_dir` to module constructors.

Usage & Operation

Q: I ran a module but got empty results — why?

A: Common causes of empty results:

1. **OPSEC mode too restrictive** — In `stealth` mode, most techniques are blocked. Check the log for `OPSEC BLOCKED` messages. Switch to `--opsec normal`.
2. **Target unreachable** — The host is down, firewalled, or the target string is wrong. Check with a manual `ping` or `nmap -Pn`.
3. **Tool not installed** — A missing optional tool produces no output for its phase. Check for `ToolNotFoundError` in output.
4. **Wrong target format** — Network/AD modules expect IPs or hostnames. Web/API modules expect URLs (`https://...`). Passing a URL to the network module or an IP to the web module won't work correctly.
5. **WAF blocking requests** — Web fuzzing (gobuster, ffuf) may be blocked by a WAF. Check `wafw00f` results in the surface phase output.
6. **No services on scanned ports** — The target has no open ports in the scanned range. In stealth mode, only a limited port set is scanned.

Q: How do I decrypt encrypted loot?

A: Encrypted loot files have a `.enc` suffix (e.g., `loot.json.enc`). The Fernet key is at `~/.reconforge/loot.key`.

```
from core.loot_manager import LootManager

plaintext_json = LootManager.load_encrypted("outputs/10.10.10.1/network/
loot.json.enc")
print(plaintext_json)
```

Or from the command line:

```
python -c "
from core.loot_manager import LootManager
print(LootManager.load_encrypted('outputs/10.10.10.1/network/loot.json.enc'))
"
```

Important: If you lose `~/.reconforge/loot.key`, encrypted loot cannot be recovered. Back up this file.

Similarly, the CredentialVault uses a separate key at `~/.reconforge/vault.key` for encrypted vault files.

Q: OPSEC mode is blocking a technique I need — how do I override it?

A: You cannot override individual OPSEC checks from the CLI. The design is intentional: OPSEC modes are safety rails.

Your options:

1. **Change the OPSEC mode:** Use `--opsec aggressive` to allow all noise levels.
2. **Run specific phases:** Use `--phases` to select only the phases you need: `--phases discovery,scanning`.
3. **Use `--dry-run` first:** See what would be blocked before committing to a mode.

The detection map (`core/detection_map.py`) defines noise levels for every technique. In `stealth` mode, only `low` noise is allowed. In `normal`, `low` and `medium`. In `aggressive`, everything is allowed.

Q: Why did a phase not run?

A: Phases can be skipped for several reasons:

1. **OPSEC restriction** — The profile for your OPSEC mode may exclude certain phases. For example, `stealth_web` only includes the `surface` phase. Check `config/profiles.yaml`.
2. **Workflow condition not met** — In workflow mode, modules run conditionally. The AD module only runs if AD services (LDAP, Kerberos, SMB) are found. The web module only runs if HTTP services are found. Check the workflow log for `Skipping <module> – condition not met`.
3. **Phase not in `--phases` list** — If you specify `--phases`, only those phases execute. Others are skipped.
4. **Opt-in required** — The web `exploit` phase and API `authorization` phase require opt-in. They only run when explicitly included in `--phases`.

Q: How are workflow conditions evaluated?

A: The `WorkflowOrchestrator` (`core/workflow_orchestrator.py`) evaluates conditions using the `WorkflowContext` — a shared data bus populated by each module's results.

Conditions are Python callables that query the context:

- `condition_ad(ctx)` → `True` if any AD service found (ldap, kerberos, smb, msrpc) or a domain is known
- `condition_web(ctx)` → `True` if any HTTP service found or a URL is known
- `condition_api(ctx)` → Same as web (HTTP service or URL)
- `condition_surface(ctx)` → `True` if there are any targets

The network module's results populate `ctx.open_ports`, `ctx.services`, and `ctx.live_hosts`. These are checked by downstream conditions.

Q: How do I resume an engagement?

A: Workflow mode saves engagement state to JSON files:

```
outputs/workflow/engagement_YYYYMMDD_HHMMSS.json
```

Resume with:

```
python reconforge.py workflow --target 10.10.10.1 \
  --resume outputs/workflow/engagement_20250321_143000.json
```

The `EngagementManager` restores:

- Engagement metadata (name, client, operator, scope)
- Timeline of all actions
- Module results
- Findings and loot summaries
- Status (paused → resumed)

Note: Resuming re-runs the entire workflow pipeline, but the engagement metadata and timeline are preserved from the previous run. Individual module results are re-executed, not cached.

Q: How do I interpret low/heuristic findings?

A: ReconForge uses a strict classification system:

Confidence	Meaning	Max Severity
confirmed	Exploited or verified	critical
high	Strong evidence of exploitability	critical
medium	Moderate evidence, needs validation	high
low	Weak evidence, manual verification needed	medium
heuristic	Pattern-based, no concrete evidence	low

If a finding has `confidence: heuristic`, it was detected by pattern matching (e.g., URL path matching, response code analysis) rather than actual exploitation. The description will include `[severity clamped: X→Y]` if the severity was reduced.

Action: Treat heuristic findings as leads for manual investigation, not confirmed vulnerabilities.

Configuration

Q: How do I customize tools.yaml?

A: `config/tools.yaml` defines every external tool's configuration. To customize:

```
tools:
  nmap:
    binary: nmap           # Binary name or full path
    required: true         # true = framework refuses to start without it
    default_timeout: 600   # Seconds before kill
    scan_profiles:
      syn_scan:
        args: "-sS --open" # Default arguments
        timeout: 600       # Profile-specific timeout
        detection: medium  # OPSEC noise level
```

Common customizations:

- **Change binary path:** Set `binary: /usr/local/bin/nmap` if the tool isn't on PATH
- **Adjust timeouts:** Increase `default_timeout` for slow networks
- **Modify scan profiles:** Change nmap arguments for your environment
- **Add new scan profiles:** Add entries under `scan_profiles`

Q: How do I customize profiles.yaml?

A: `config/profiles.yaml` defines OPSEC-aware scan profiles. Each profile controls:

- **Timing:** nmap timing template, scan delay, max retries
- **Allowed noise levels:** Which detection levels are permitted
- **Phase restrictions:** Which phases run in a given mode
- **Tool toggles:** Which tools are enabled/disabled
- **Module-specific options:** Per-module configuration

Example: Create a custom profile for a specific client:

```
profiles:
  client_acme:
    description: "Custom profile for Acme Corp – no UDP, limited SMB"
    opsec_mode: normal
    timing:
      nmap_timing: T3
      scan_delay: "100ms"
    scanning:
      port_range: "1-10000"
      udp_scan: false
    enumeration:
      enum4linux: true
      smb_scripts: false
    allowed_noise_levels:
      - low
      - medium
```

The `ProfileLoader` (`core/profile_loader.py`) resolves profiles by name. Module-specific profiles (e.g., `stealth_ad`, `normal_web`) take priority over base profiles.

Q: How do I add custom wordlists?

A: For the web module, use `--wordlist` :

```
python reconforge.py web --target https://target.com --wordlist /path/to/custom-wordlist.txt
```

For the API module:

```
python reconforge.py api --target https://api.target.com/v1 --wordlist /path/to/api-wordlist.txt
```

The wordlist is passed to the underlying fuzzing tools (gobuster, ffuf). If no wordlist is specified, the tools use their own defaults.

Q: How do I adjust timeouts?

A: Three levels of timeout control:

1. **Global CLI timeout:** `--timeout 1200` (applies to all commands in the run)
2. **Tool-level timeout:** Set in `config/tools.yaml` under `default_timeout`
3. **Profile-level timeout:** Set per scan profile in `config/tools.yaml`

The Runner (`core/runner.py`) uses the most specific timeout: per-command override > tool default > global default.

Output & Findings

Q: How do I understand confidence levels?

A: Confidence indicates how reliable the evidence is:

- **confirmed** — The vulnerability was verified (e.g., actual data extracted via SQLi)
- **high** — Strong indicators but not fully exploited (e.g., known-vulnerable version detected)
- **medium** — Moderate evidence requiring follow-up (e.g., suspicious response pattern)
- **low** — Weak signal requiring manual investigation (e.g., generic error message)
- **heuristic** — Pattern-only detection with no concrete evidence (e.g., parameter name suggests IDOR)

The FindingsManager (`core/findings_manager.py`) enforces severity capping based on confidence. Heuristic findings can never exceed `low` severity.

Q: How do I interpret severity?

A: Severity reflects potential impact:

- **critical** — Requires `confirmed` or `high` confidence. Immediate exploitation possible with significant impact.
- **high** — Requires at least `medium` confidence. Significant security issue.
- **medium** — Moderate impact or moderate evidence.
- **low** — Minimal impact or weak evidence (includes all heuristic findings).
- **info** — Informational, no direct security impact.

Q: Where do I find output files?

A: All output follows this structure:

```
outputs/<sanitized_target>/<module>/
├── raw/           # Unprocessed tool output
├── parsed/        # Structured parsed data
├── findings.json  # Machine-readable findings (JSON array)
├── findings.md    # Human-readable findings (Markdown)
├── loot.json      # Credentials, tokens, shares, services
├── session.md     # Timestamped session notes
└── commands.log   # All commands executed in this run
```

Target names are sanitized: `/` → `_`, `:` → `_`, spaces → `_`. So `https://app.target.com` becomes `https__app.target.com`.

AD module additionally produces:

- `attack_paths.md` — Identified attack chains
- `ad_summary.md` — AD environment summary
- `quick_report.md` — Executive-level report

Q: How is loot organized?

A: Loot items have a `loot_type` field:

Type	Description	Example
credential	Username:password pair	admin:P@ssw0rd
hash	Password hash	NTLM hash from SAM
token	Authentication token	JWT, Bearer token
user	Enumerated username	jsmith
share	Accessible network share	\\10.10.10.1\public
service	Discovered service/version	Apache/2.4.51

Each item includes `source` (which tool found it), `module` (which module), `confidence`, and `metadata` (type-specific details).

Q: How are session notes structured?

A: `session.md` is a timestamped timeline generated by the `NotesManager` (`core/notes_manager.py`). Entries are categorized:

-  **phase** — Phase start/end markers
-  **finding** — Notable finding discovered
-  **command** — Command execution notes
-  **general** — General observations

Troubleshooting

Q: A command timed out — what do I do?

A: The Runner kills commands that exceed their timeout. You'll see:

```
Command timed out after 600s: nmap -sV -sC 10.10.10.1
```

Options:

1. **Increase timeout:** `--timeout 1200`
2. **Reduce scope:** Scan fewer ports (`--phases discovery` only, or configure port range in profiles)
3. **Check network:** Slow networks cause timeouts. Verify connectivity manually.
4. **Use stealth mode:** Stealth uses T1/T2 timing which is inherently slower — expect timeouts for large scans. Increase timeout accordingly.

Q: A tool failed with a non-zero exit code

A: Tool failures are logged but don't crash the framework. The `RunResult` captures `returncode`, `stdout`, and `stderr`. Check `commands.log` in the output directory for the full command and `session.md` for context.

Common causes:

- Tool doesn't support the arguments used (version mismatch)
- Target rejected the connection
- Tool-specific errors (e.g., nuclei template errors)

Q: A parser threw an error

A: Parsers extract structured data from raw tool output. If a tool's output format changes (due to version updates), parsers may fail. The framework catches parser exceptions and logs warnings — results for that tool will be empty, but the module continues.

Check `raw/` for the actual tool output and file a bug if the parser doesn't handle it.

Q: A module crashed — what happens?

A: In standalone mode (`python reconforge.py <module>`), a module crash propagates to the CLI and exits with code 1.

In workflow mode, the behavior depends on the step's `critical` flag:

- **Non-critical step:** Logged as failed, workflow continues to next step
- **Critical step:** Workflow aborts immediately

The engagement state is saved regardless, and can be resumed.

Q: The workflow was interrupted — how do I recover?

A: If you hit Ctrl+C during a workflow:

1. The current step is marked as `failed` with error "Interrupted by user"
2. The workflow summary is saved to `outputs/workflow/workflow_YYYYMMDD_HHMMSS.json`
3. The engagement state is saved to `outputs/workflow/engagement_YYYYMMDD_HHMMSS.json`

Resume with:


```
python reconforge.py workflow --target 10.10.10.1 \
  --resume outputs/workflow/engagement_YYYYMMDD_HHMMSS.json
```

Advanced Usage

Q: Can I create custom modules?

A: Yes. Each module follows a consistent architecture:

```
modules/<name>/
├── <name>_module.py  # Main orchestrator class
├── base.py           # Module base class
├── tools/            # Tool wrapper classes
├── parsers/          # Output parser classes
└── phases/           # Phase orchestration classes
```

Your module class needs:

- `MODULE_NAME` — String identifier
- `VALID_PHASES` — List of phase names
- `run(phases=None, **kwargs)` — Main entry point returning a results dict
- Integration with core managers: `FindingsManager`, `LootManager`, `NotesManager`, `OutputManager`, `AttackWorkflow`

See existing modules (e.g., `modules/network/network_module.py`) for the pattern.

Q: Can I add custom tools?

A: Yes. Create a tool wrapper in `modules/<module>/tools/`:

1. Define the tool in `config/tools.yaml` with binary path, timeout, and detection levels
2. Create a wrapper class that uses `Runner.run()` to execute commands
3. Create a corresponding parser in `modules/<module>/parsers/`
4. Integrate into the relevant phase class

Tool wrappers should:

- Check tool availability with `Runner.check_tool()`
- Build commands as `list[str]` (not shell strings)
- Return `RunResult` objects
- Log via the module's `ReconLogger`

Q: Can I add custom phases?

A: Yes. Create a phase class in `modules/<module>/phases/`:

1. The phase class should accept references to the module's tools, parsers, and core managers
2. Implement a `run()` method that orchestrates tool execution and result parsing
3. Register the phase in `VALID_PHASES` on the module class
4. Integrate into the module's `run()` method

Q: How do I integrate ReconForge with other tools?

A: Several integration points:

- **Input:** Pass targets via `--target` CLI argument
- **Output:** Parse `findings.json` (structured JSON) or `loot.json` for downstream tools
- **Workflow API:** Use `WorkflowOrchestrator` programmatically:

```
from core.workflow_orchestrator import WorkflowOrchestrator

wf = WorkflowOrchestrator.targeted(
    targets=["10.10.10.1"],
    modules=["network", "ad"],
    opsec_mode="normal",
    output_base="outputs",
)
summary = wf.run()
```

- **Findings API:** Use `FindingsManager` directly:

```
from core.findings_manager import FindingsManager

fm = FindingsManager()
# ... add findings ...
fm.save_json(Path("custom_findings.json"))
fm.save_markdown(Path("custom_findings.md"))
```

- **BloodHound integration:** The AD module's bloodhound phase collects data compatible with the BloodHound GUI
- **Credential vault:** Export credentials for use in other tools:

```
from core.credential_vault import CredentialVault

vault = CredentialVault()
vault.load(Path("outputs/workflow/vault_20250321_143000.json"))
vault.export_usernames(Path("usernames.txt"))
vault.export_hashes(Path("hashes.txt"))
vault.export_passwords(Path("passwords.txt"))
```